Dated ETF Allocation Disclosures: A Relational Design and Data-Access Layer*

Taimoor Shahzad
University of Victoria
Victoria, Canada

M. Shahmeer Ather Kakwan
University of Victoria
Victoria, Canada

Mason Li
University of Victoria
Victoria, Canada

Abstract

Exchange-traded fund (ETF) allocation percentages need dates, category meanings and source evidence to support comparison. Our relational design separates customer holdings from provider disclosures and scopes categories by classification edition or captured-source context. A Flask interface supports search, creation, batch updates, deletion and dated portfolio views over a fully synthetic dataset. Twelve tables, SQL constraints and transaction validation preserve the modeled rules; 74 automated tests and six analytical queries exercise the implementation. Code, setup and evaluation records: https://github.com/Pyrex111/CSC501_Data_Modeling_Project.

1 Relational Design

Twelve relations and thirteen foreign keys implement the revised ER model. `Holding` represents the dated Portfolio–ETF association; `AllocationEntry` associates a disclosure snapshot with a category and weight. Holding and allocation dates remain independent. Page three lists all relations, attributes and constraints.

Every table has an autogenerated integer `Id`. Candidate keys preserve customer numbers, portfolio references within each customer, provider registration within a jurisdiction, unique ISINs and tickers within an exchange. Source captures use URL plus retrieval timestamp. Category uniqueness includes dimension, context and label: context identifies a known scheme edition or an unresolved captured block. Matching labels do not establish equivalence.

`ETFProvider` models registered companies; `Customer` models individual bank customers. Names describe these records rather than identify them. A portfolio belongs to one customer, and its dated holdings refer to ETFs. Each disclosure groups one dated sector or geography allocation block from a retained capture. Entries refer to categories rather than repeating unscoped labels. A known classification stores its authority, name and edition; unresolved categories remain local to the capture and locator. Thus two "Technology" labels need not denote the same investment grouping, even when their reported percentages happen to agree.

SQLite STRICT tables, scalar checks and foreign keys enforce types, dates, fractions and references [2, 3]. Four triggers protect category/snapshot dimensions. The transaction validator checks required identifiers/entries, contexts and Complete totals within 0.005 of one. This configurable fraction tolerance is provisional, not source-calibrated. Raw SQL can bypass transaction checks. File existence is checked only when `capture_root` is supplied; it does not establish authenticity.

*Project repository: https://github.com/Pyrex111/CSC501_Data_Modeling_Project

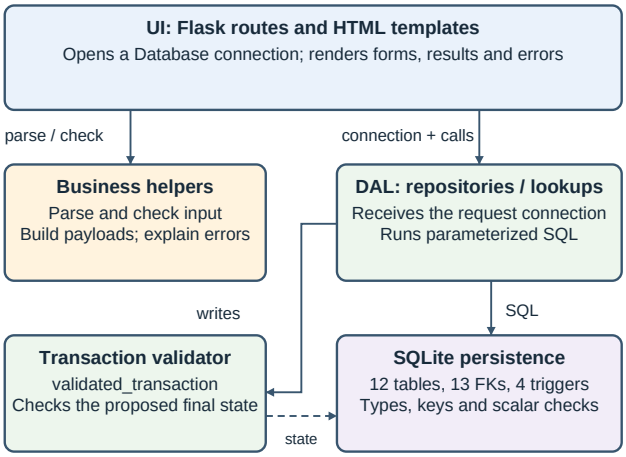


Figure 1: Routes open/pass connections and call helpers and DAL functions separately. Repository writes invoke the final-state validator.

2 Queries and Application

2.1 Data and SELECT queries

All identities and observations are fictional. Three readable synthetic PDFs match the seed; URLs use example.invalid. The Python loader and `seed.sql` yield identical dumps. Every table is populated: three ETFs, six identifiers, thirteen categories, five Complete disclosures and seventeen entries; each Complete total is one.

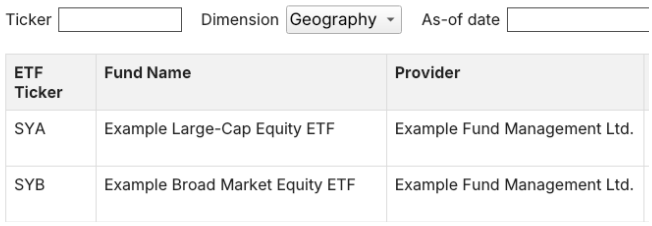
Q1 joins geography entries to funds and sources (8 rows). Q2 ranks Financials allocations within one fictional scheme edition (2). Q3 groups by snapshot with `COUNT` and `SUM` to show coverage/totals (5). Q4 separates observation, publication and retrieval dates (5). Q6 traces entries to category context and retained capture (17).

Q5 returns five *diagnostic candidates*: holding weight multiplied by candidate geography allocation. For example, $0.6 \times 0.985 = 0.591$ is not validated exposure. Look-through also requires explicit holding/disclosure dates, coverage/basis checks and compatible category meanings. The query retains dates and contexts instead of silently aggregating them.

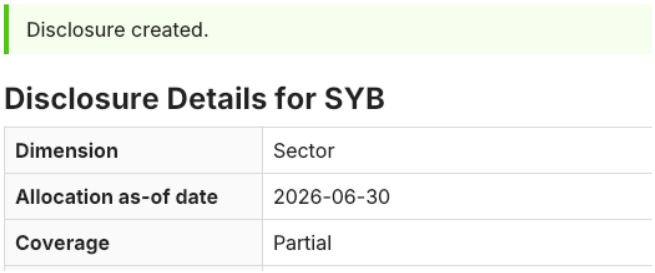
2.2 Writes and reproducibility

INSERT, UPDATE and DELETE bind user values through parameters and use the transaction validator. Creation saves a snapshot and entries atomically; batch edits validate the combined result. Invalid totals or missing batch entries roll back every change. Deletion must preserve an entry and any Complete-total requirement; successful Partial deletion is demonstrated in Figure 2.

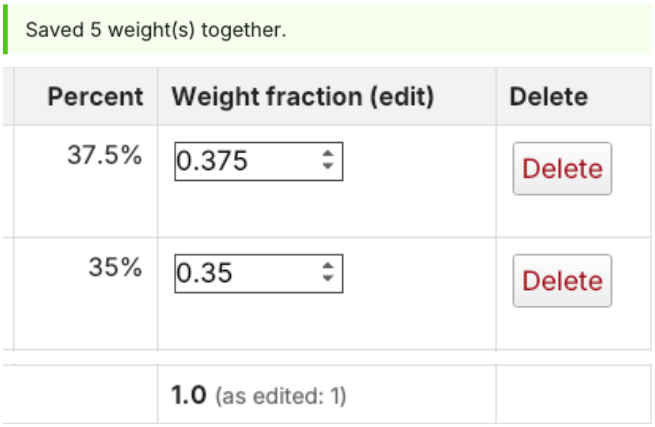
(a) Search: Geography filter and results



(b) Create: saved Partial disclosure



(c) Update: batch saved; total remains 1



(d) Delete: two entries remain; total 0.7

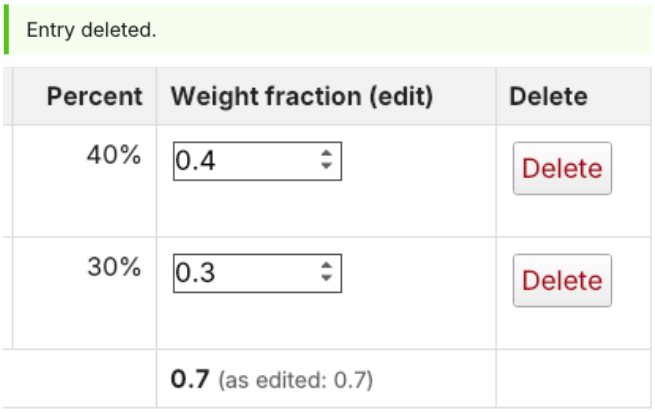


Figure 2: Unchanged screenshot excerpts: search, successful creation, a coherent batch update and deletion from a Partial disclosure. Intervening rows are omitted; update and delete are separate examples. All nine full-resolution captures, including rejected writes, provenance and portfolio views, are in `supplements/ui_screenshots/`. All data are fictional.

After installing dependencies, `quickstart.py` builds/seeds a missing database; `-serve` starts the UI. Later runs validate/reuse data; only `-reset` replaces it, after a successful temporary build. The 74 tests comprise 33 schema, 11 integration and 30 UI-route tests. Six-query output, seed equivalence, source-PDF consistency and reuse/reset were also verified. Synthetic checks validate behavior, not provider accuracy.

2.3 Interface and validation

Users search by ticker, dimension and allocation date; inspect classification and source evidence; create disclosures; batch-edit weights; and delete entries. Portfolio selection displays a holding date alongside separately dated disclosures, without unsupported aggregation. The prototype operates on preloaded funds, sources and categories.

Routes call business helpers and DAL functions separately (Figure 1). Helpers reject invalid dates, nonfinite or out-of-range fractions, duplicate categories and empty disclosures. SQL and transaction checks apply before commit. Rejected creates/updates retain input and show plain messages: the recorded total 1.04 is rejected, while a coherent batch saves at 1.0. Full screenshots document each operation and rejection.

Recorded write examples (screenshots 04–09):

Operation / proposed state	Result
Create Partial; total 0.8	Saved
Create Complete; total 0.8	Rejected
Batch update; total 1.04	Rejected
Batch update; total 1.0	Saved
Delete breaking Complete total	Rejected
Delete from Partial example	Saved; total 0.7

2.4 AI assistance and evaluation

OpenAI ChatGPT/Codex assisted with modeling, code, tests, figures and prose [1]. Taimoor's supplied critique covers keys, dates, contexts and tolerance; Mason reports independent integrated reruns. Project evaluation records distinguish human review from assistant-executed checks. Personal browser testing is not inferred.

References

[1] OpenAI. 2026. ChatGPT/Codex: AI assistance with code, modeling, figures and text. https://chatgpt.com/
[2] SQLite Development Team. [n. d.]. *SQLite Foreign Key Support.* https://www.sqlite.org/foreignkeys.html
[3] SQLite Development Team. [n. d.]. *STRICT Tables.* https://www.sqlite.org/stricttables.html

3 Relations and constraints

Every relation has an autogenerated `Id` INTEGER primary key. Foreign-key columns are INTEGER; `WeightFraction` is REAL; other columns are TEXT. Tables are STRICT. All columns are NOT NULL except those marked ?; required text is nonempty, and identity text has no surrounding spaces. All FKs reference `Id`, with UPDATE/DELETE RESTRICT. **UK** denotes uniqueness; conditional UKs apply only to their stated rows.

Customer
`Id, CustomerNumber, GivenName, FamilyName, Email?`
UK: CustomerNumber. Names and Email are not unique.

Portfolio
`Id, CustomerId, PortfolioReference, PortfolioName, BaseCurrency`
FK: CustomerId → Customer. UK: (CustomerId, PortfolioReference).

Holding
`Id, PortfolioId, ETFId, WeightFraction, HoldingAsOfDate`
FKs: PortfolioId → Portfolio; ETFId → ETF. UK: (PortfolioId, ETFId, HoldingAsOfDate).

ETFProvider
`Id, LegalName, RegistrationNumber, RegistrationJurisdiction, WebsiteUrl?`
UK: (RegistrationJurisdiction, RegistrationNumber). LegalName is descriptive, not unique.

ETF
`Id, ETFProviderId, FundName, DomicileCountryCode, BaseCurrency, InceptionDate?`
FK: ETFProviderId → ETFProvider. Business identity is supplied by ETFIdentifier; FundName is not unique. Each saved ETF requires at least one identifier (transaction rule).

ETFIdentifier
`Id, ETFId, IdentifierScheme, IdentifierValue, ExchangeContext?`
FK: ETFId → ETF. IdentifierScheme ∈ {ISIN, Ticker}. Values are uppercase and nonempty. ISIN: UK (IdentifierScheme, IdentifierValue), ExchangeContext is NULL; value has 12 uppercase alphanumeric characters, two initial letters and a final digit. Ticker: UK (IdentifierScheme, IdentifierValue, ExchangeContext); exchange is required, uppercase and nonempty. ISIN checksum is not checked.

Common scalar rules. Fractions are in $[0, 1]$. Currency codes have three uppercase letters and domicile codes two. Dates are valid `YYYY-MM-DD`. Source timestamps use UTC: `YYYY-MM-DDTHH:MM:SS.ffffffZ`. Optional text is NULL when unknown and nonempty when present. Code formats do not verify external registry membership.

SourceDocument
`Id, ETFProviderId, Url, RetrievedAt, PublishedAt?, CapturedFileRef, DocumentTitle?`
FK: ETFProviderId → ETFProvider. UK: (Url, RetrievedAt). CapturedFileRef identifies a retained capture; the transaction validator checks file existence within the supplied project root.

AllocationDimension
`Id, DimensionName`
UK: DimensionName; DimensionName ∈ {Sector, Geography}.

ClassificationScheme
`Id, SchemeAuthority, SchemeName, Version?, DefinitionUrl?`
Conditional UK: (SchemeAuthority, SchemeName, Version). Applies only WHERE Version IS NOT NULL. Unknown editions retain NULL Version and no asserted complete business key.

AllocationCategory
`Id, AllocationDimensionId, ClassificationSchemeId?, Label, Code?, ContextKey`
FKs: AllocationDimensionId → AllocationDimension; ClassificationSchemeId → ClassificationScheme (optional). UK: (AllocationDimensionId, ContextKey, Label). ContextKey is generated from known authority/name/version or from captured URL/retrieval time/block locator/dimension. Unresolved contexts are local to the disclosure; matching labels/codes do not prove equivalence.

AllocationSnapshot
`Id, ETFId, AllocationDimensionId, SourceDocumentId, AllocationAsOfDate, BasisNote, CoverageStatus, SourceLocator`
FKs: ETFId → ETF; AllocationDimensionId → AllocationDimension; SourceDocumentId → SourceDocument. UK: (ETFId, AllocationDimensionId, AllocationAsOfDate, SourceDocumentId, SourceLocator). CoverageStatus ∈ {Complete, Partial, Unknown}. SourceLocator distinguishes dated allocation blocks/bases within a capture.

AllocationEntry
`Id, AllocationSnapshotId, AllocationCategoryId, WeightFraction`
FKs: AllocationSnapshotId → AllocationSnapshot; AllocationCategoryId → AllocationCategory. UK: (AllocationSnapshotId, AllocationCategoryId).

Cross-relation rules. SQL triggers require matching category and snapshot dimensions on entry writes and parent dimension edits. Before COMMIT, the transaction validator requires an entry per saved snapshot, validates used category contexts and checks Complete totals within 0.005 of 1 (inclusive, configurable). Partial/Unknown totals and sampled holding totals need not equal 1. Parent minimums, contexts, totals and file checks require the application validator; raw SQL COMMIT can bypass them.